// DEEP DIVE

Network Address Translation (NAT)

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How millions of private devices share a handful of public IP addresses — translation tables, port forwarding, carrier-grade NAT, traversal techniques, and the protocols that still break because of it.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	What NAT is, why it exists, and how translation actually works
3 — 5	Static, Dynamic & PAT — with a full worked translation table and port forwarding
6 — 7	Carrier-Grade NAT and the security trade-offs of NAT
8 — 9	NAT traversal (STUN/TURN/ICE) and the protocols that break under NAT

How to use this guide: read start to finish for the full picture, or jump to section 4 if you just need the worked translation-table example. Every diagram is numbered and referenced directly in the surrounding text.

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

What Is NAT & Why It Exists

1.1 THE PROBLEM: TOO MANY DEVICES, TOO FEW ADDRESSES

Network Address Translation (NAT) is a technique that rewrites IP address information in packet headers as traffic passes through a router, allowing multiple devices on a private network to share a single public IP address. It was standardized in 1994 (RFC 1631) as a stopgap for one very specific problem: **IPv4 only has about 4.3 billion addresses**, and the number of internet-connected devices blew past that number years ago.

Without NAT, every laptop, phone, smart bulb, and thermostat in the world would need its own globally unique public IP address. NAT sidesteps this by letting an entire home, office, or even an entire ISP's customer base share one (or a small handful of) public addresses, while every device internally uses a private address from a reserved range.

> **Private IP ranges (RFC 1918):** `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16` are reserved for private networks and are never routed on the public internet. NAT is the bridge between this private space and the public internet.

1.2 WHAT NAT ACTUALLY DOES

At its core, a NAT-enabled router keeps a **translation table** that maps private (internal) addresses to public (external) ones. When a packet leaves the private network, the router rewrites the source IP (and often the source port) to its own public address before forwarding it. When a reply comes back, the router looks up the translation table, rewrites the destination back to the original private address, and forwards it inward.

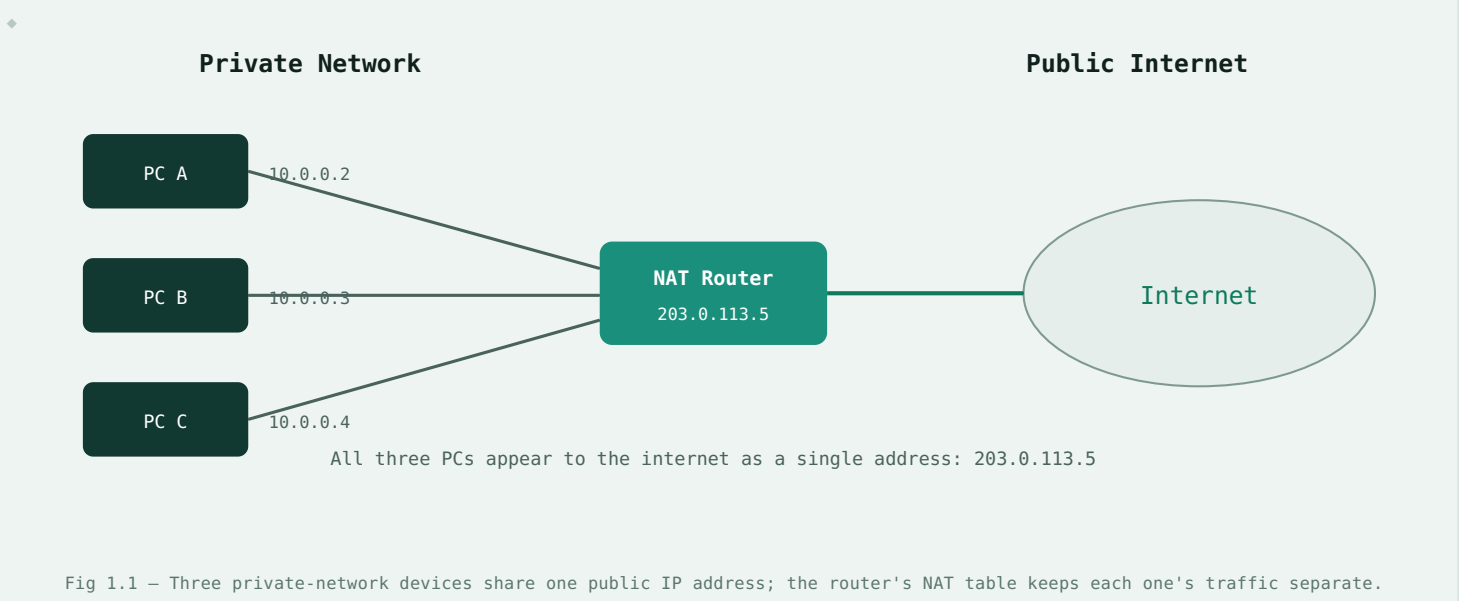

Fig 1.1 — Three private-network devices share one public IP address; the router's NAT table keeps each one's traffic separate.

1.3 WHY NAT STUCK AROUND EVEN AFTER IPV6

IPv6, with its 128-bit address space, was designed specifically to make NAT unnecessary — there are enough addresses for every device on Earth many times over. But NAT persisted for reasons beyond address scarcity:

- **Security by obscurity** — internal devices are not directly addressable from the internet, which acts as a rough first line of defense.
- **Renumbering flexibility** — internal addressing can change without needing new public addresses or ISP coordination.
- **Inertia** — IPv6 adoption has been slow, and existing NAT-based infrastructure works well enough that migration pressure stayed low.

// SECTION 2

How NAT Works

2.1 THE TRANSLATION TABLE

Every NAT router maintains a table of active translations. Each entry typically records the internal (private) IP and port, the external (public) IP and port it's mapped to, the destination it's talking to, and the protocol in use. This table is what lets return traffic find its way back to the correct internal device.

2.2 STEP-BY-STEP: A SINGLE CONNECTION

1. PC A (`10.0.0.2:51000`) sends a packet to a web server at `93.184.216.34:443`.
2. The NAT router intercepts the packet, replaces the source address with its own public IP and a chosen source port — e.g. `203.0.113.5:40001` — and records this mapping in its table.
3. The rewritten packet travels the internet and reaches the web server, which believes it's talking directly to `203.0.113.5:40001`.
4. The server's reply is addressed to `203.0.113.5:40001`. The NAT router receives it, looks up the table, finds the match, and rewrites the destination back to `10.0.0.2:51000` before forwarding it inward.

> **Key insight:** the internal device never sees its packets being rewritten. NAT is completely transparent to the private-side host — as far as PC A knows, it's talking directly to the web server.

2.3 WHY PORT NUMBERS MATTER

A single public IP address has 65,536 possible port numbers per protocol. Since most home NAT setups use only one public IP, the router relies on **port numbers** to keep multiple internal devices' connections distinct — this is technically called **PAT (Port Address Translation)** and is covered in Section 3. Without unique source ports, the router would have no way to tell PC A's traffic apart from PC B's once both are rewritten to the same public IP.

// SECTION 3

Types of NAT

3.1 STATIC NAT

A **one-to-one, permanent** mapping between one private IP and one public IP. Every packet from that internal host always translates to the same external address, and anyone on the internet can reach it at that fixed address. Used when an internal server (mail server, web server) needs to be consistently reachable from outside.

3.2 DYNAMIC NAT

Maps private IPs to public IPs from a **pool** of available addresses, assigned on a first-come, first-served basis. If a company owns a block of 10 public IPs and has 40 internal hosts, up to 10 of those hosts can have internet access simultaneously — the mapping isn't fixed and is released when the connection ends. Less common today since pools of public IPs are expensive and PAT solves the same problem more efficiently.

3.3 PAT / NAPT (PORT ADDRESS TRANSLATION)

The type used in almost every home router. **Many private IPs share one public IP**, disambiguated by port number rather than address. This is why PAT is sometimes called **NAT overload** — a single public address can theoretically support tens of thousands of simultaneous connections, one per unique port mapping.

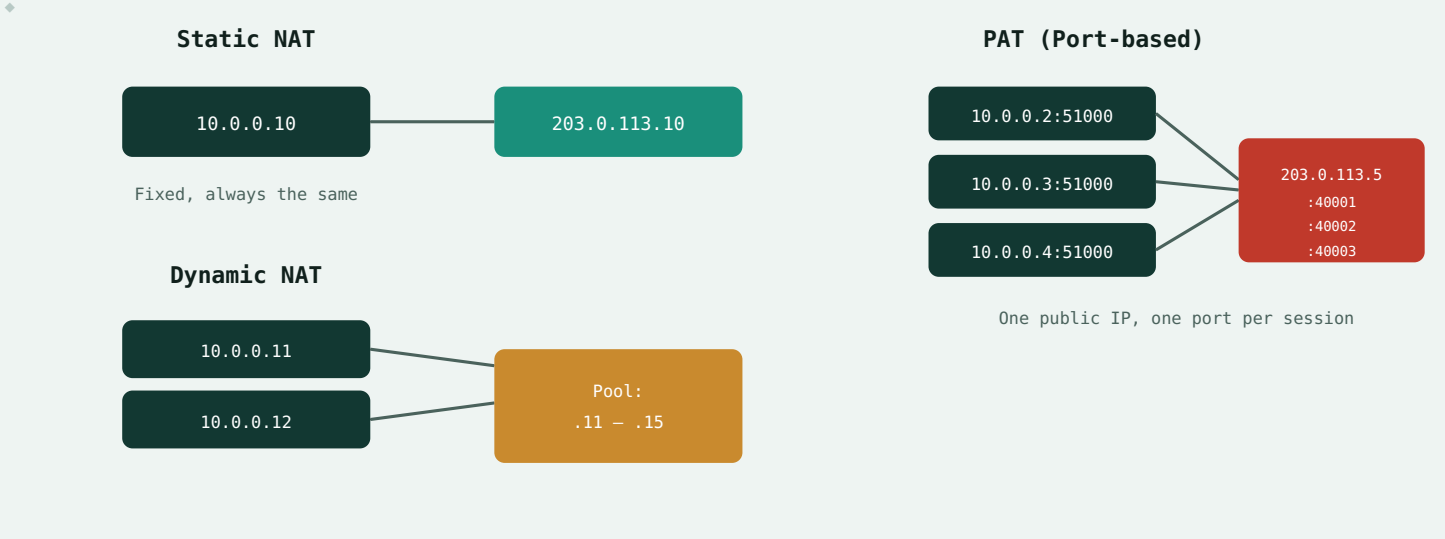

Fig 3.1 — Static NAT (fixed 1:1), Dynamic NAT (pooled 1:1), and PAT (many:1 via port numbers) side by side.

3.4 QUICK COMPARISON

TYPE	MAPPING	PUBLIC IPS NEEDED	TYPICAL USE
Static NAT	1 private : 1 public (fixed)	One per host	Servers that must be reachable from outside
Dynamic NAT	1 private : 1 public (pooled)	A pool, fewer than hosts	Legacy enterprise internet access
PAT / NAPT	Many private : 1 public (by port)	Just one	Home routers, almost everything today

// SECTION 4

Worked Example: A Full Translation Table

Consider a home router with public IP `203.0.113.5` serving three devices that all happen to open connections around the same time. Here's what its NAT (PAT) table looks like mid-session:

INTERNAL IP:PORT	EXTERNAL IP:PORT	DESTINATION	PROTOCOL
10.0.0.2:51000	203.0.113.5:40001	93.184.216.34:443	TCP
10.0.0.3:52200	203.0.113.5:40002	142.250.premium:443	TCP
10.0.0.4:53311	203.0.113.5:40003	8.8.8.8:53	UDP
10.0.0.2:51001	203.0.113.5:40004	93.184.216.34:443	TCP

> **Notice the last two rows:** PC A (`10.0.0.2`) has two simultaneous connections to the same server — perhaps two browser tabs. Each gets its own external port (`40001` and `40004`) even though the internal source port only differs slightly. The external port is what makes each connection uniquely trackable.

4.1 WHAT HAPPENS WHEN A REPLY ARRIVES

A packet arrives at the router addressed to `203.0.113.5:40002`. The router scans its table, finds the matching row, and learns that this traffic actually belongs to `10.0.0.3:52200`. It rewrites the destination address accordingly and forwards the packet onto the private network. If no matching row exists — meaning this connection was never initiated from inside — the packet is simply dropped, which is exactly the behavior that gives NAT its incidental firewall-like protection (more on this in Section 7).

4.2 TABLE ENTRY LIFETIME

Translation table entries aren't permanent. TCP entries are typically held for a few hours after the connection appears idle, and UDP entries (which have no explicit "close" signal) often expire in as little as 30 seconds of inactivity — a detail that matters a great deal for real-time protocols like VoIP, covered in Section 9.

// SECTION 5

Port Forwarding

NAT's default behavior only allows outbound-initiated connections back in — which is great for security but a problem if you're hosting something (a game server, a security camera's web UI, a self-hosted app) that needs to accept **inbound** connections from the internet. **Port forwarding** is a manual, permanent exception carved into the NAT table: a rule that says "any inbound traffic to my public IP on port X should always go to this specific internal device on port Y," regardless of whether that internal device asked for it.

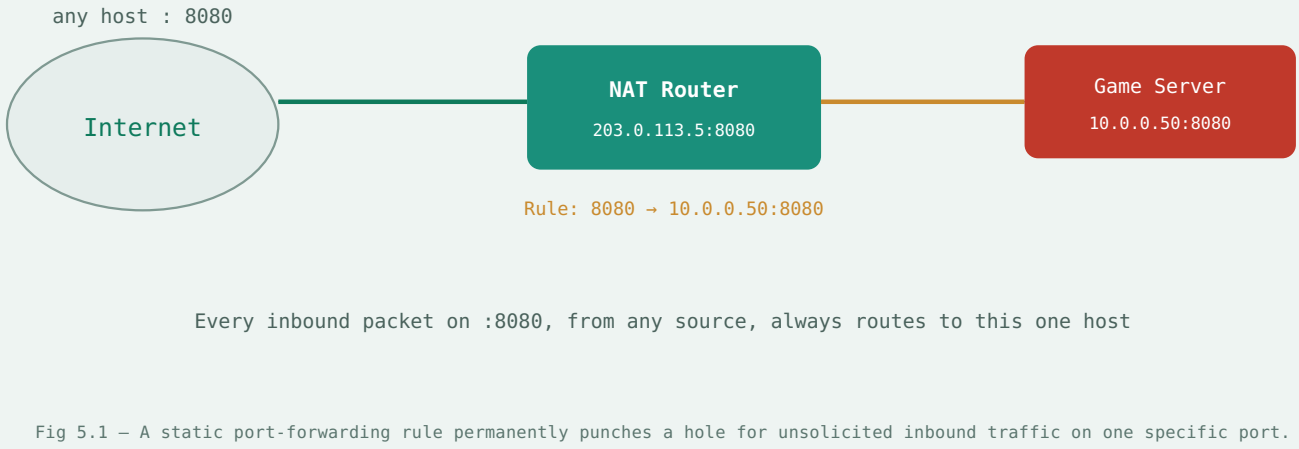

Fig 5.1 — A static port-forwarding rule permanently punches a hole for unsolicited inbound traffic on one specific port.

5.1 PORT FORWARDING VS. AUTOMATIC NAT

ASPECT	STANDARD NAT (PAT)	PORT FORWARDING
Direction	Outbound-initiated only	Allows unsolicited inbound
Table entry	Temporary, created automatically	Permanent, configured manually
Security exposure	Internal hosts hidden by default	Deliberately exposes one host:port
Typical use	Everyday browsing, apps	Game servers, self-hosted services, cameras

Security note: every forwarded port is a deliberate hole in your network's default protection. Forward only what you need, to the specific internal host that needs it, and keep that host patched — it's now directly reachable from anyone on the internet who finds it.

5.2 UPNP & NAT-PMP: AUTOMATIC PORT FORWARDING

Manually configuring port forwarding rules doesn't scale for consumer devices, so protocols like **UPnP (Universal Plug and Play)** and its successor **NAT-PMP / PCP** let applications request their own forwarding rules programmatically — this is how a game console or torrent client can open the ports it needs without you touching the router's admin page. It's convenient, but also a known attack surface: malware inside the network can use the same mechanism to open ports without your knowledge.

// SECTION 6

Carrier-Grade NAT (CGNAT)

6.1 NAT, BUT ONE MORE LAYER UP

Home NAT solves address sharing within one household. But ISPs face the same problem at a much larger scale: they often don't have enough public IPv4 addresses for every subscriber either. **Carrier-Grade NAT (CGNAT)**, also called Large-Scale NAT, applies the same translation concept at the ISP level — hundreds or thousands of customers, each already behind their own home NAT, share a single ISP-side public IP.

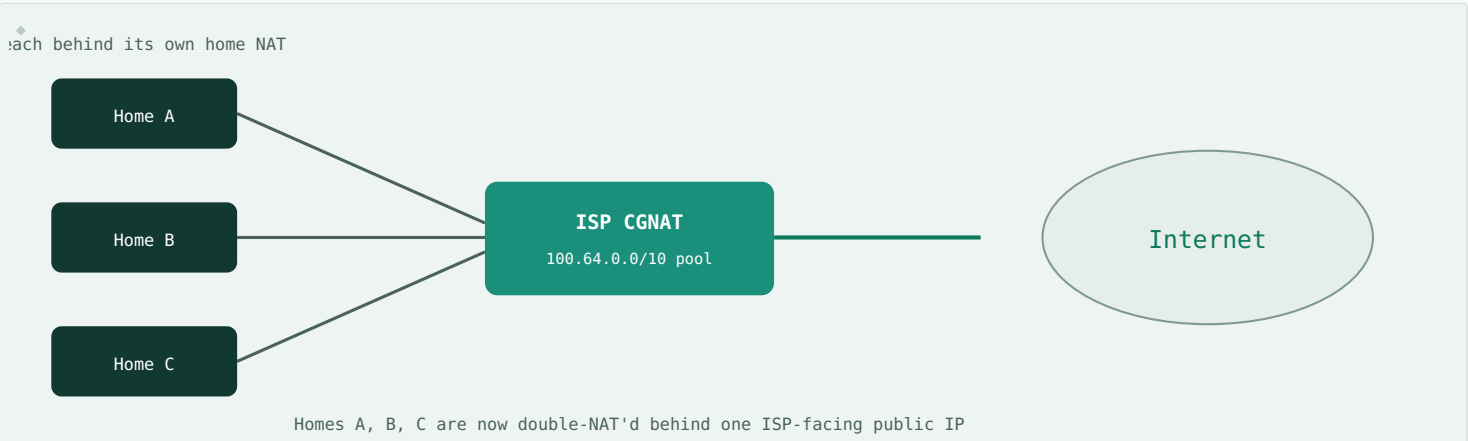

Fig 6.1 — CGNAT stacks a second NAT layer at the ISP, sharing one public IP across many already-NAT'd households.

6.2 THE TRADE-OFFS

CGNAT extends the life of IPv4 without every subscriber needing IPv6, but it comes at a real cost:

- **Port forwarding stops working** — customers no longer control the actual public-facing NAT layer, so they can't forward ports for game servers, cameras, or self-hosted services the way they could with a dedicated public IP.
- **Shared reputation risk** — if another customer sharing your CGNAT IP gets it blacklisted (spam, abuse), your traffic can be affected too.
- **Harder troubleshooting & geolocation** — a single public IP now represents many unrelated households, complicating abuse tracking, geolocation accuracy, and some anti-fraud systems.
- **Extra latency and a single point of failure** — one more hop, one more piece of stateful infrastructure the ISP must scale and keep available.

The IETF reserved a dedicated block, `100.64.0.0/10`, specifically for CGNAT use (RFC 6598) — separate from the RFC 1918 private ranges — so that a customer's own home NAT and the ISP's CGNAT layer don't collide over the same address space.

// SECTION 7

NAT & Security Implications

7.1 THE ACCIDENTAL FIREWALL

NAT was never designed as a security feature, but its default behavior — dropping any inbound packet that doesn't match an existing outbound-initiated table entry — happens to behave a lot like a basic stateful firewall. Internal hosts are simply unreachable from the outside unless something inside asked first, or a port-forwarding rule says otherwise. This is genuinely useful protection, but it's a side effect, not a designed guarantee, and shouldn't be relied on as a substitute for an actual firewall.

> **Common misconception:** "I'm behind NAT, so I'm safe." NAT hides your internal topology and blocks unsolicited inbound connections, but it does nothing to stop outbound-initiated threats — phishing, malware calling home, or drive-by downloads all originate as outbound connections that NAT was designed to allow through cleanly.

7.2 WHERE NAT ACTIVELY COMPLICATES SECURITY

- **Breaks end-to-end traceability** — a single public IP can represent one device or thousands (under CGNAT), making IP-based logging, blocking, and geolocation far less reliable.
- **Interferes with IPsec** — NAT rewriting packet headers conflicts with IPsec's integrity checks, which is why **NAT-Traversal (NAT-T)** exists as a workaround (Section 9).
- **Encourages port-forwarding sprawl** — as more devices need inbound access, more forwarding rules accumulate, each one a deliberate exception to NAT's default protection.
- **UPnP/NAT-PMP abuse** — malware already inside the network can silently open forwarding rules for itself, turning an internal compromise into an internet-exposed one.

7.3 NAT IS NOT A FIREWALL — USE BOTH

Modern routers pair NAT with an actual stateful firewall, explicit ACLs, and often intrusion detection. The right mental model: NAT's blocking behavior is a convenient bonus, while a dedicated firewall is the actual, intentional security control — the two should be configured and reasoned about separately.

// SECTION 8

NAT Traversal: STUN, TURN, ICE & Hole Punching

Peer-to-peer applications — video calls, VoIP, online games, file-sharing — have a fundamental problem: both endpoints are usually behind NAT, and neither one can accept unsolicited inbound connections. A set of standard techniques, collectively called **NAT traversal**, solves this without requiring manual port forwarding.

8.1 STUN — Session Traversal Utilities for NAT

A **STUN server** sitting on the public internet simply tells a client what its own public IP and port look like from the outside — information the client can't determine on its own since NAT rewrites it in transit. Both peers query STUN, exchange these public-facing addresses (usually via a signaling channel like SIP or a game server), and then attempt to connect **directly** to each other.

8.2 Hole Punching

The trick that makes direct peer-to-peer connections work through NAT: both peers simultaneously send outbound packets toward each other's public IP:port. Each side's own NAT sees this as "we initiated an outbound connection" and opens a temporary return path — even though neither side ever received an inbound request first. The nearly-simultaneous outbound packets "punch" matching holes in both NAT tables at once.

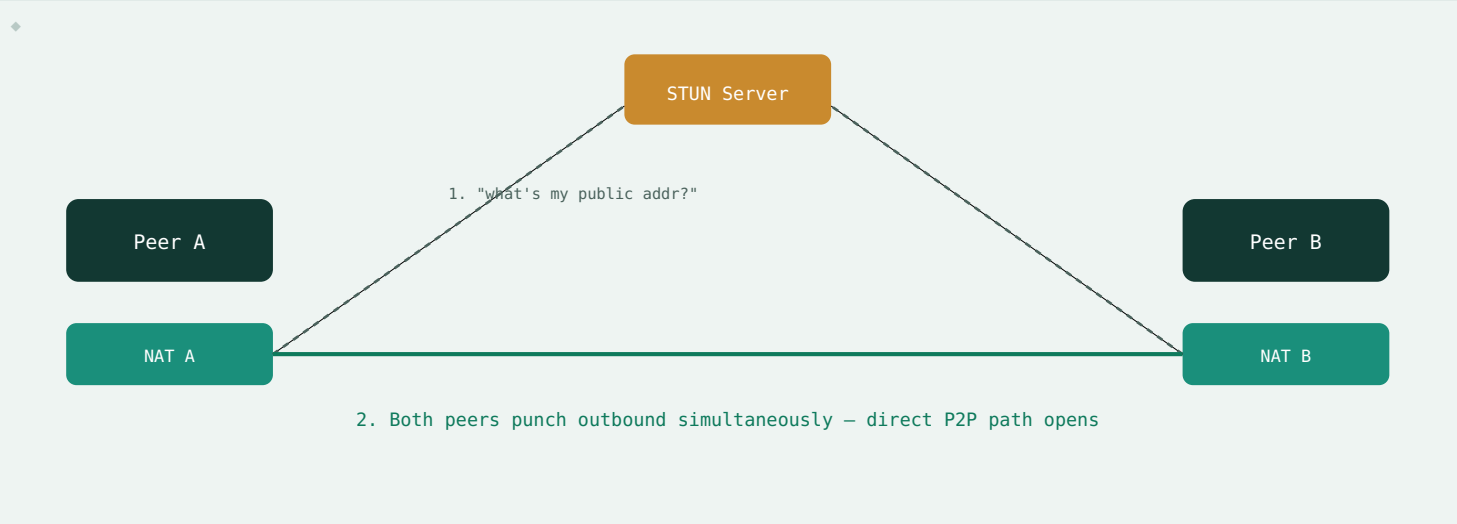

Fig 8.1 — STUN reveals each peer's public-facing address; simultaneous outbound packets then punch a direct path through both NATs.

8.3 TURN — When Hole Punching Fails

Some NAT implementations (especially **symmetric NAT**, which assigns a different external port for every distinct destination) make hole punching unreliable. **TURN (Traversal Using Relays around NAT)** is the fallback: a relay server on the public internet that both peers connect to normally (outbound-initiated, so NAT-friendly), and which simply forwards traffic between them. It always works, but adds latency and server cost since all traffic now flows through a third party instead of directly between peers.

8.4 ICE — Tying It Together

ICE (Interactive Connectivity Establishment) is the framework that WebRTC and most modern VoIP/video systems actually use: it gathers all possible connection paths (direct local address, STUN-discovered public address, TURN relay address), tries them in order of preference, and automatically falls back to TURN only if every direct option fails. This is why video calls usually connect fast and directly, but still work even across the most restrictive NAT/firewall combinations.

TECHNIQUE	ROLE	TRAFFIC PATH
STUN	Discover your own public IP:port	Direct, once path is established
Hole Punching	Open simultaneous NAT mappings on both sides	Direct peer-to-peer
TURN	Relay traffic when direct P2P isn't possible	Via relay server (indirect)
ICE	Coordinate and pick the best of the above	Whichever succeeds first

// SECTION 9

Protocols That Break Under NAT

Most protocols carry all the addressing information they need inside the IP/TCP/UDP headers, which NAT rewrites transparently without issue. A handful of older or address-sensitive protocols break this assumption by embedding IP addresses or port numbers **inside the application-layer payload** — data NAT doesn't know to rewrite, since it normally only touches packet headers.

9.1 FTP (ACTIVE MODE)

Classic FTP's active mode has the client tell the server, in the payload of a control message, which IP and port to connect back to for the data transfer. If that IP is a private address, the server tries to connect to an address that doesn't exist on the public internet, and the transfer fails. The common fix is an **Application-Layer Gateway (ALG)** — a NAT feature that specifically understands FTP and rewrites the embedded address too — or simply using **passive mode** FTP, which avoids the problem by having the client initiate both connections.

9.2 SIP & VOIP

SIP (Session Initiation Protocol) embeds IP addresses and ports in its SDP payload to negotiate where audio/video streams should be sent. Behind NAT, this embedded address is the private one, useless to the far end. This is precisely the problem STUN/TURN/ICE (Section 8) were built to solve, and most modern VoIP and video-calling software has this handled automatically.

9.3 IPSEC & NAT-TRAVERSAL (NAT-T)

IPsec's AH (Authentication Header) mode cryptographically signs parts of the IP header itself — so if NAT rewrites that header in transit, the signature no longer matches and the packet is rejected as tampered. **NAT-T** works around this by encapsulating IPsec traffic inside an extra UDP layer (typically port 4500), so NAT only ever touches the outer UDP header it's designed to rewrite, leaving the protected inner packet untouched.

9.4 QUICK REFERENCE: PROTOCOL IMPACT

PROTOCOL	WHAT BREAKS	COMMON FIX
FTP (active)	Embedded IP/port in control payload	ALG, or use passive mode
SIP / VoIP	Embedded address in SDP payload	STUN / TURN / ICE
IPsec (AH mode)	Header rewrite invalidates signature	NAT-Traversal (NAT-T), UDP encapsulation
Online gaming (P2P)	No inbound path for peer connections	UPnP/NAT-PMP, port forwarding, or relay servers

// SECTION 10

Quick Reference & Glossary

TERM	DEFINITION
NAT	Network Address Translation — rewrites IP/port info so multiple devices can share public IP addresses.
PAT / NAPT	Port Address Translation — many private IPs share one public IP, distinguished by port number.
Translation Table	The router's record mapping internal IP:port pairs to external IP:port pairs.
Port Forwarding	A manual, permanent rule allowing unsolicited inbound traffic on a specific port to reach one internal host.
CGNAT	Carrier-Grade NAT — an ISP-level NAT layer shared across many customers, stacked on top of their home NAT.
UPnP / NAT-PMP	Protocols letting applications automatically request their own port-forwarding rules.
STUN	Protocol that tells a client its own public-facing IP:port as seen from outside its NAT.
TURN	A relay server that forwards traffic between peers when a direct connection isn't possible.
ICE	Framework that tries direct, STUN, and TURN paths in order and picks the first that works.
Hole Punching	Simultaneous outbound packets from both peers that open matching temporary NAT mappings.
Symmetric NAT	A NAT variant that assigns a different external port per destination, making hole punching unreliable.
NAT-T	NAT-Traversal — encapsulates IPsec in UDP so NAT can rewrite headers without breaking packet integrity checks.
ALG	Application-Layer Gateway — a NAT feature that understands and rewrites addresses embedded in specific protocols' payloads.

End of guide. NAT is one of those technologies that was meant to be temporary and became permanent infrastructure — understanding its translation model makes everything from "why can't I host a server" to "why does my video call sometimes lag through a relay" make a lot more sense.